CS-UY 4513 In-Class Activity #6 Deliverable

Rails Relational Tables, Foreign Keys, and Nested Routes

Complete the following table of team member names/emails/presence today in class. List ALL team members, not just those present today.

Name	NetID (@nyu.edu)	Present (Y/N)
Taso Lafazanidis	al9049	Y
David Jojan	dj2583	Y
Zachary Kublalsingh	zjk2012	Y
Brandon Gao	bg2650	Y
Bryan Wang	bw2809	Y
Joshua Woo	jjw9119	Y
Gaurav Poddar	gp2610	Y

- Clone the repo at: https://github.com/NYU-CSE-Software-Engineering/in-class-references-and-nested-routes
- Deliverables due today (uploaded to your team repo (docs/):
    - This document completed as a PDF
- Deliverables due today (upload to Brightspace):
    - **`db/schema.rb`** with all seven tables and the foreign key declarations visible.
    - Source listing of all models (all seven **`app/models/*.rb`** files).
    - Nested routes listing: **`rails routes | grep -E "stages|tracks|performances|tickets"`**

Festival Schema

Rails relational tables, foreign keys, and nested routes (ESaaS Chapter 5)

The Scenario

Your team is building the database side of a music festival app: seven tables, the models on top of them, and the nested routes. You'll work in a fresh practice app, so nothing here can break your class project.

This is Chapter 5 practice: foreign keys, **`belongs_to`** and **`has_many`**, **`has_many :through`**, and nested routes. Everything gets written by hand, which means no **`rails generate`** and no scaffolds. Typing the migrations and models yourself is how the syntax sticks.

The Data Model

Pair	Parent	Child (holds the foreign key)	Relationship
A	**`Festival`** `(name, starts_on, city)`	**`Stage`** `(name, capacity, festival_id)`	A **`Festival`** has many **`Stages`**
B	**`Artist`** `(name, genre)`	**`Track`** `(title, length_sec, artist_id)`	An **`Artist`** has many **`Tracks`**
C	**`Attendee`** `(name, email)`	**`Ticket`** `(tier, price_cents, attendee_id, festival_id)`	An **`Attendee`** has many **`Tickets`**; a **`Festival`** also has many **`Tickets`**
Integrator		**`Performance`** `(starts_at, stage_id, artist_id)`	**`Stages`** and **`Artists`** are many-to-many through **`Performances`**

Column types in your models are up to you, but I recommend choosing reasonable ones (**`string`**, **`integer`**, **`date`**, **`datetime`**, and so on).

Roles

The letters A, B, and C name the three resource sets, even when one person works a set alone. Find your team size below.

Team size	A: Festival → Stage	B: Artist → Track	C: Attendee → Ticket	Performance
5	Pair	One student (solo)	Pair	The solo student starts the skeleton as soon as **`Artist`** and **`Track`** are done. The whole team joins at minute 50.
6	Pair	Pair	Pair	The whole team builds it together from minute 50.
7	Pair	Pair	Pair	The seventh student is the **Integrator**. Start the migration and model skeleton at minute 10, then pull in **`Stage`** and **`Artist`** as the pairs finish.

On a team of 5, set B (Artist → Track) is the smallest, so the solo student should finish first. Use any spare time to review a pair's models and help check their rails console tests.

Pair C has the trickiest table because **`Ticket`** has two foreign keys.

Setup

1. One team member should clone the provided starter repo: **`git clone https://github.com/NYU-CSE-Software-Engineering/in-class-references-and-nested-routes.git`**
2. That member should manage the repo. All members can write files and share with the repo manager.
3. Create a branch for each pair, such as **`pair-a-festivals`**.
4. Run **`bundle install`** and confirm **`rails db:migrate`** runs with no errors.

Migration file names

To make sure parent tables are created before the children that reference them, use exactly these file names in **`db/migrate/`**:

Order	File name
1	`20260101000100_create_festivals.rb`
2	`20260101000200_create_stages.rb`

Order	File name
3	20260101000300_create_artists.rb
4	20260101000400_create_tracks.rb
5	20260101000500_create_attendees.rb
6	20260101000600_create_tickets.rb
7	20260101000700_create_performances.rb

Inside each file, the class name must match the file name (for example, **`CreateFestivals`**) and inherit from **`ActiveRecord::Migration[7.1]`**. Check your starter app's **`Gemfile.lock`** for the right Rails version number.

A syntax refresher (different domain)

Here is a migration and models for a library app. Use it as a pattern, not something to copy.

```
# db/migrate/20250101000100_create_books.rb
class CreateBooks < ActiveRecord::Migration[7.1]
  def change
    create_table :books do |t|
      t.string  :title
      t.integer :pages
      t.references :library, foreign_key: true
      t.timestamps
    end
  end
end

# app/models/library.rb
class Library < ApplicationRecord
  has_many :books, dependent: :destroy
end

# app/models/book.rb
class Book < ApplicationRecord
  belongs_to :library
  validates :title, presence: true
end
```

Timeline

Minutes	Phase	What to do
0-10	**Plan**	Read this handout, divide into pairs, and sketch an ER diagram on paper or on a classroom whiteboard.
10-35	**Pair phase**	Write migrations and models. Run **`rails db:migrate`**. Prove it works in rails console.
35-50	**Routes phase**	Add nested routes for your pair's resources and study the output of rails routes.
50-70	**Integration**	Merge branches, then build Performance.
70-80	**Wrap-up**	Assemble deliverables and answer the reflection questions.

Phase 1: Plan (0-10 min)

Before writing code, answer these as a team:

- **For each of the four relationships, which table holds the foreign key?**

  Festival-Stage->Stages holds the foreign key for festival_id. There are many stages for a festival.

  Artist-Track -> Track holds the foreign key for artist. There are many tracks for an artist.

  Attendee,festival-ticket -> Tickets holds the foreign key for attendee and festival. There are many tickets for an attendee and festival.

  Stage,Artist-Performance-> Performance holds the foreign keys for stages and artists to break a many to many relationship. There are many performances for a stage and an artist.

- **Why does the foreign key belong on the "many" side and not the "one" side?**

  The foreign key belongs to the many sides as many of the child objects point to one of the parent objects.

- **Which tables must be migrated first, and why?**
  Tables with no foreign keys (parent tables) must be migrated first as they are independent to the rest of the schema. Then tables that reference those models are merged.

Phase 2: Pair Phase (10-35 min)

Each pair does the following for its parent and child resources:

1. Write the two migration files using **`t.references ..., foreign_key: true`** on the child.
2. Write the two model files with **`has_many / belongs_to`**.
3. Add **`dependent:`** on the **`has_many`** side. Choose between **`:destroy`**, **`:nullify`**, and **`:restrict_with_error`**, and be ready to justify your choice.
4. Add at least one validation to each model (for example, **`presence: true`**).
5. Run rails **`db:migrate`** and look at **`db/schema.rb`**. Find your foreign key lines near the bottom.
6. Verify in rails console:

```
f = Festival.create!(name: "Sunset Fest", starts_on: Date.today, city:
"Austin")
f.stages.create!(name: "Main Stage", capacity: 5000)
f.stages.count
Stage.first.festival   # should return the festival
```

Pair C note: **`Ticket`** belongs to both an **`Attendee`** and a **`Festival`**. Add **`belongs_to`** for both, and a matching **`has_many :tickets`** on both parent models.

Commit your work to your pair's branch before the routes phase.

Phase 3: Routes Phase (35-50 min)

Edit **`config/routes.rb`**. Each pair adds nested routes for its resources.

Pair	Required routes
A	Stage nested under **`Festival`**, giving URLs such as **`/festivals/1/stages`**
B	Track nested under **`Artist`** using shallow nesting (**`shallow: true`**)
C	Ticket nested under **`Attendee`**, and also reachable through **`Festival`**

Controllers do not need to exist. rails routes work without them, so you can focus on route shape.

For each pair, **`run rails routes | grep <resource>`** and answer in a comment at the top of your routes block:

- **What URL path would you use to list the children of one parent?**

Pair A: GET /festivals/:festival_id/stages

Pair B: GET /artists/:artist_id/tracks

Pair C: GET /attendees/:attendee_id/tickets (or /festivals/:festival_id/tickets)

Performance: GET /stages/:stage_id/performances

GET /artists/:artist_id/performances

- **Which route parameter holds the parent's id (:festival_id, :id, and so on)?**

The parameter is named after the parent resource followed by _id.

- **Pair A:** :festival_id
- **Pair B:** :artist_id
- **Pair C:** :attendee_id and :festival_id
- **Performance:** :stage_id, :artist_id

- **Pair B only: What changed in the output when you used `shallow: true`? Which routes are still nested, and which are not? Why does that make sense?**

Pair B:

- **What changed:** The routes for individual tracks were un-nested from the artist, while the collection routes remained nested.
- **Still nested:** index,new, and create (e.g., /artists/:artist_id/tracks).
- **Not nested:** show, edit, update, and destroy (e.g., /tracks/:id).
- **Why it makes sense:** Since every track has a unique, globally identifying primary key (id), the database doesn't need the artist_id to look up, or delete a specific track.

- **Pair C only: Tickets are nested in two places. Why might both be justified? Which would you use to show one attendee's purchase history, and which to show a festival's ticket sales?**

Pair C:

- **Why both are justified:** A ticket acts as a join model between an attendee and a festival. Viewing tickets through the lens of either parent represents a completely different, valid business use case.
- **Attendee's purchase history:** You would use the routes nested under the attendee (/attendees/:attendee_id/tickets).
- **Festival's ticket sales:** You would use the routes nested under the festival (/festivals/:festival_id/tickets).

Phase 4: Integration (50-70 min)

Step 1: Merge branches (about 5 min)

Merge all pair branches into **`main`** (or your team's integration branch). You may hit conflicts in **`config/routes.rb`** and definitely in **`db/schema.rb`**.

Warning: do not hand-merge **`schema.rb`**. Resolve the migration files and **`routes.rb`**, then delete **`db/schema.rb`**, run **`rails db:drop db:create db:migrate`**, and let Rails regenerate it.

Step 2: Build Performance (about 15 min)

Performance is the join model that connects a **`Stage`** to an **`Artist`** at a particular time.

1. Write **`20260101000700_create_performances.rb`** with **`starts_at`** and two foreign keys.
2. Write the **`Performance`** model with the right **`belongs_to lines`**.
3. Update **`Stage`** and **`Artist`** so each has **`has_many :performances`** and a **`has_many :artists`** / **`has_many :stages`** using **`through: :performances`**.
4. Add nested routes so these URLs exist:

   ```
   /stages/:stage_id/performances
   /artists/:artist_id/performances
   ```

5. Verify in rails console:

   ```
   stage = Stage.first
   artist = Artist.first
   Performance.create!(stage: stage, artist: artist, starts_at:
   1.day.from_now)
   stage.artists        # should include the artist
   artist.stages        # should include the stage
   ```

Phase 5: Wrap-up (70-80 min)

Prepare your team's deliverables.

Deliverables (one submission per team)

1. **`db/schema.rb`** with all seven tables and the foreign key declarations visible.
2. Source listing of all models (all seven **`app/models/*.rb`** files).
3. Nested routes listing:

    a. `rails routes | grep -E "stages|tracks|performances|tickets"`

4. Reflection answers (a few sentences each):

    a. **In each relationship, which table holds the foreign key, and why that table?**

    The table on the "many" (child) side of the relationship always holds the foreign key (e.g., stages holds festival_id, tracks holds artist_id). A relational database record can only contain a single value per column; placing the foreign key on the child allows many unique child records to point back to the single parent ID.

    b. **What happens to a Festival's Stages when the Festival is destroyed? Where in your code did you specify that behavior?**

    The stages are automatically deleted. This behavior is specified in app/models/festival.rb via the dependent: :destroy option on the has_many :stages declaration.

    a. **Why does Performance need to be its own model with its own table, rather than a bare join table with no model?**
    A bare join table (has_and_belongs_to_many) can only store the two foreign keys. The Performance concept requires storing additional data specific to that relationship—specifically the starts_at timestamp. Because it holds its own attributes, it requires a full model using has_many :through.

We don't expect every team to finish everything. Completing roughly 90% in the time available is a good result.

Common Pitfalls

- Migrations out of order. A child table can't reference a parent that doesn't exist yet. Check your file names.
- Only half an association. Remember **`belongs_to`** on the child and **`has_many`** on the parent.
- Singular vs. plural. It's **`resources :stages`**, but **`belongs_to :stage`**.
- Forgetting to migrate. Models won't work in the console until rails **`db:migrate`** has run.
- Hand-merging **`schema.rb`**. Don't. Regenerate it.
- Stuck on a failing migration? Run **`rails db:rollback`**, fix the file, and migrate again. Check **`rails db:migrate:status`** to see where you are.